Тестовое задание: C# WPF + CMake - Просмотрщик покерных логов

Цель: Разработать desktop-приложение на C#/WPF под Windows 10/11, которое через UI позволяет указать папку, запускает фоновый поток для сканирования JSON-файлов, парсит данные о покерных раздачах и отображает их в иерархическом интерфейсе.

Технические требования:

Параметр	Требование
Язык	C# (.NET 6 или .NET 8)
UI-фреймворк	WPF (без сторонних UI-библиотек)
Паттерн UI	MVVM (обязательно)
Многопоточность	Только явные `Thread`. Использование `async/await`, `Task`, `BackgroundWorker` или `ThreadPool` запрещено.
Парсинг JSON	`System.Text.Json` или `Newtonsoft.Json`
Сборка	CMake → генерация `.sln` и `.csproj`, открываемых в VS 2019–2022
ОС	Windows 10/11 (x64)
VCS	Git с корректным `.gitignore`

Функциональные требования:

1. Выбор директории через UI

- На форме должно быть поле ввода/кнопка для выбора папки сканирования (рекомендуется **`FolderBrowserDialog`**).
- Путь должен быть сохранён в ViewModel и передан в сервис сканирования.

2. Алгоритм работы

- Пользователь указывает папку и нажимает кнопку `Начать сканирование`.
- Кнопка блокируется, на UI меняется статус: `Сканирование...`
- Запускается отдельный `Thread`, который:

1. Рекурсивно находит все `.json` файлы в указанной директории.
2. Считывает и парсит каждый файл. Формат данных в файле:

json

```
1. [
2.  {
3.   "HandID": 123456789,
4.   "TableName": "Berlin #01",
5.   "Players": ["Jack", "Henry", "Daniel"],
6.   "Winners": ["Jack"],
7.   "WinAmount": "1 000,00$"
8.  }
9. ]
```

3. Корректно пропускает битые/невалидные файлы без падения приложения.

- Обновление UI: по мере сбора данных или после полного сканирования поток должен безопасно передавать данные в UI-поток.
- Завершение работы: после обхода всех файлов поток завершается, кнопка разблокируется, статус на UI меняется на ✅ `Готово (N файлов обработано)` или ❌ `Ошибка: ...`

3. Интерфейс (WPF)

- Левая панель: список уникальных `TableName` (сортировка по алфавиту).
- Средняя панель: при выборе `TableName` отображается список связанных `HandID`.
- Правая панель: при выборе `HandID` выводятся `Players`, `Winners` и `WinAmount`.
- При отсутствии данных или пустом выборе отображаются понятные placeholder-сообщения.

Архитектура и качество кода

- Разделение на слои: `Views` ↔ `ViewModels` ↔ `Services` ↔ `Models`.
- Многопоточность: явное создание `Thread`, передача параметров, обработка `ThreadAbortException`/`InterruptedException` при необходимости.
- Синхронизация с UI: использование `Dispatcher.Invoke`/`Dispatcher.BeginInvoke` или `SynchronizationContext`. Прямое изменение UI-элементов из фонового потока запрещено.
- Безопасность коллекций: обновление `ObservableCollection<T>` только из UI-потока, либо использование `BindingOperations.EnableCollectionSynchronization`.
- Обработка исключений в фоновом потоке, логирование в `Debug`/`Console`, graceful degradation.
- Чистый, читаемый код, соблюдение SOLID и naming conventions.

Сборка через CMake

- В корне репозитория должен лежать `CMakeLists.txt`.
- Команды:

  bash

  ```
  1 cmake -B build -G "Visual Studio 17 2022"  # или "Visual Studio 16 2019

  2 cmake --build build --config Release
  ```

- CMake должен генерировать рабочий `.sln`, который открывается в Visual Studio 2019–2022 без ручных правок.
- *Примечание:* Допускается использование нативной поддержки C# в CMake 3.27+ либо кастомных `add_custom_target`, вызывающих `dotnet build`. Главное – итоговый `.sln` должен быть валидным и воспроизводимым.

Git-репозиторий

- Инициализировать репозиторий и настроить `.gitignore` под .NET/WPF/Visual Studio/CMake.
- В репозитории не должно быть: `bin/`, `obj/`, `.vs/`, `build/`, `*.user`, `*.suo`, `packages/` и т.п.
- История коммитов должна отражать ход разработки (минимум 3–5 осмысленных коммитов).

- В корне обязателен `README.md` с инструкцией по сборке, запуску и примером тестовых данных.

Формат сдачи

- Ссылка на Git-репозиторий (публичный или с предоставленным доступом).
- В `README.md` указать:
  - Версию .NET SDK
  - Команды для конфигурации и сборки
  - Как запустить приложение
  - Краткое описание выбранного подхода к синхронизации потоков и обновления коллекций
- Ожидаемое время выполнения: до 2 календарных дней.

Критерии оценки

Критерий	Вес
Соответствие функциональным требованиям и алгоритму работы	25%
Многопоточность и синхронизация (`Thread`, `Dispatcher`, безопасность коллекций, отсутствие `async/await`)	30%
Архитектура (MVVM, разделение ответственности, тестируемость)	20%
Качество кода, обработка ошибок, читаемость	15%
Корректная интеграция CMake → VS Solution	5%
Git-гигиена, `.gitignore`, документация	5%

Бонусные задания (необязательные, но приветствуются)

- Реализовать `FileSystemWatcher` для автоматического обновления данных без перезапуска сканирования (в отдельном потоке).
- Добавить поиск/фильтрацию по `TableName` или `HandID`.
- Написать 2–3 unit-теста для парсера/сервиса.
- Локализация интерфейса (RU/EN).

*Примечание для кандидата:
Задание фокусируется на понимании работы с потоками в WPF, корректной маршалировке данных в UI-поток и управлении состоянием интерфейса. Если какой-то пункт вызывает вопросы, задавайте их до начала разработки.
Удачи! 🚀